

NAME:KRISHITHA .G

REG NO:192524451

Course code:CSA0613

Course title: Design and Analysis of Algorithms

CO3_ASSESSMENT TOOL 2 _ CASE STUDY BASED ASSESSMENT

Title: Climate Simulation Model Using Divide and Conquer Matrix Multiplication (Strassen's Algorithm)

Aim:

To implement matrix multiplication using the Divide and Conquer approach with Strassen's Algorithm, improve the performance of large-scale matrix operations used in climate simulation models, and analyze its time complexity.

Software Used:

- Python

Algorithm:

- Start.
- Read the two square matrices of equal order.
- Check whether the matrix size is suitable for recursive division.
- Divide each matrix into four equal-sized submatrices.
- Compute the seven intermediate matrices using Strassen's Algorithm.
- Combine the intermediate results to form the final product matrix.
- Display the resultant matrix.
- Analyze the time complexity as $O(n^{2.81})$.
- Stop.

Source Code (Python)

```
1 ▾ def add(A, B):
2     n = len(A)
3     return [[A[i][j] + B[i][j] for j in range(n)] for i in
              range(n)]
4
5 ▾ def subtract(A, B):
6     n = len(A)
7     return [[A[i][j] - B[i][j] for j in range(n)] for i in
              range(n)]
8
9 ▾ def split(matrix):
10    n = len(matrix)
11    mid = n // 2
12
13    A11 = [row[:mid] for row in matrix[:mid]]
14    A12 = [row[mid:] for row in matrix[:mid]]
15    A21 = [row[:mid] for row in matrix[mid:]]
16    A22 = [row[mid:] for row in matrix[mid:]]
17
18    return A11, A12, A21, A22
19
20 ▾ def strassen(A, B):
21    n = len(A)
22
23 ▾    if n == 1:
24        return [[A[0][0] * B[0][0]]]
```

```

25
26     A11, A12, A21, A22 = split(A)
27     B11, B12, B21, B22 = split(B)
28
29     M1 = strassen(add(A11, A22), add(B11, B22))
30     M2 = strassen(add(A21, A22), B11)
31     M3 = strassen(A11, subtract(B12, B22))
32     M4 = strassen(A22, subtract(B21, B11))
33     M5 = strassen(add(A11, A12), B22)
34     M6 = strassen(subtract(A21, A11), add(B11, B12))
35     M7 = strassen(subtract(A12, A22), add(B21, B22))
36
37     C11 = subtract(add(add(M1, M4), M7), M5)
38     C12 = add(M3, M5)
39     C21 = add(M2, M4)
40     C22 = add(subtract(add(M1, M3), M2), M6)
41
42     new_matrix = []
43
44     for i in range(len(C11)):
45         new_matrix.append(C11[i] + C12[i])
46
47     for i in range(len(C21)):
48         new_matrix.append(C21[i] + C22[i])
49
50     return new_matrix
51
52
53 # Driver Program
54 A = [
55     [1, 2],
56     [3, 4]
57 ]
58
59 B = [
60     [5, 6],
61     [7, 8]
62 ]
63
64 result = strassen(A, B)
65
66 print("Resultant Matrix:")
67 for row in result:
68     print(row)

```

OUTPUT

Output

Resultant Matrix:

[19, 22]

[43, 50]

=== Code Execution Successful ===

Analysis

Strassen's Algorithm improves matrix multiplication by using 7 recursive multiplications instead of 8, reducing the time complexity from $O(n^3)$ to $O(n^{2.81})$. This makes it more efficient for large-scale climate simulation models, though it requires additional memory for recursion and temporary matrices.

Documentation

Input:

- Two square matrices of equal order.

Output:

- Product matrix obtained using Strassen's Matrix Multiplication.

Time Complexity:

- Conventional Matrix Multiplication: $O(n^3)$
- Strassen's Matrix Multiplication: $O(n^{2.81})$

Space Complexity:

- $O(n^2)$ (due to recursive calls and temporary matrices)
- Machine learning
- Large-scale simulations

Result:

The implementation of Strassen's Matrix Multiplication successfully multiplies two matrices using the divide-and-conquer approach. Compared to the conventional method, it reduces computational time for large matrices, making it an efficient solution for climate simulation models.